

Présentation

Dans le cadre du cours **IFT-2935** à l'Université de Montréal, notre équipe a conçu le projet **Système de Gestion de Salle d'Attente Hospitalière**.

L'objectif principal de ce projet est de développer une application web complète et fonctionnelle permettant de gérer les inscriptions des patients dans les hôpitaux de Montréal, d'afficher les données en temps réel sur une carte interactive, et d'optimiser l'évaluation et la prise en charge des patients par le personnel infirmier.

- **Remarque importante:** Nous avons utilisé un ORM pour le bonus.

Donc, les **réponses aux questions 1-4 de la section 5 sont directement implémentées en backend** (package **repository**), vous pouvez vous fier au rapport ainsi qu'à l'application et constater que toutes les fonctionnalités y sont présentes. Nous avons donc inclus cette partie dans le rapport.

Liens Youtube

1. [Vidéo de présentation du projet](#)
2. [Si nécessaire, vidéo tutoriel de 2 minutes pour lancer le projet](#)

Technologies utilisées

Pour construire une application moderne, fiable et bien structurée, nous avons utilisé une pile technologique adaptée au développement web full-stack, ainsi que plusieurs outils d'architecture et de collaboration.

Backend

- **Java** : Langage principal utilisé pour la logique métier, reconnu pour sa stabilité et son intégration avec des frameworks robustes.
- **Spring Boot** : Framework Java moderne utilisé pour concevoir l'API REST, gérer la sécurité, les transactions, les services et la persistance.
- **Spring Data JPA (Hibernate)** : ORM puissant pour interagir avec la base de données sans avoir à écrire du SQL brut.
- **Spring Security** : Sécurité intégrée avec gestion des rôles, authentification et autorisations.

Base de données

- **PostgreSQL** : Système relationnel open-source, fiable, performant, offrant des fonctionnalités avancées de transactions, d'indexation et de requêtage SQL.

Frontend

- **Svelte** : Framework JavaScript compilé pour créer des interfaces ultra-rapides avec une gestion d'état réactive.
- **TypeScript** : Superset de JavaScript avec typage statique, utilisé pour améliorer la fiabilité du code.
- **Leaflet + OpenStreetMap** : Bibliothèque de cartographie interactive pour afficher les hôpitaux sur une carte dynamique.

Outils de modélisation et de gestion

- **Draw.io** : Pour la création du diagramme entité-association.
 - **FigJam** : Pour les brainstormings et la planification de l'interface.
 - **GitHub** : Pour la gestion de version et le travail collaboratif.
 - **Docker** : Pour l'orchestration des conteneurs backend, frontend et base de données.
-

Développement et architecture

Étapes du projet

1. Modélisation EA : Conception d'un modèle entité-association complet, conforme à la réalité hospitalière.
 2. Passage au schéma relationnel : Application des règles de transformation relationnelle.
 3. Initialisation du backend avec Quarkus, rapidement abandonné au profit de Spring Boot à cause de limitations techniques.
 4. Création des entités, repositories, services et contrôleurs selon un modèle en couches.
 5. Définition des DTOs pour découpler le frontend de la structure interne.
 6. Développement du frontend Svelte : composant par composant.
 7. Mise en place de la carte Leaflet et des stores Svelte.
 8. Débogage, itérations, intégration complète en Docker.
-

Migration Quarkus → Spring Boot

Nous avons initialement commencé avec Quarkus pour sa légèreté, mais avons rapidement rencontré des limites :

- Gestion difficile de JPA Panache,
- Intégration laborieuse avec Spring Security,
- Manque de support ou de documentation dans certaines situations concrètes.

Spring Boot nous a offert un environnement plus stable, documenté et modulaire.

Architecture logique (backend)

Notre backend suit l'architecture classique :

- `@RestController` : endpoints HTTP (validation, routes, statuts de réponse).
 - `@Service` : logique métier, orchestration, transactions.
 - `@Repository` : accès aux données via Spring Data JPA (CRUD + requêtes personnalisées).
 - `@Entity` : objets persistés (Patient, EvaluationMedicale, Hopital...).
-

Utilisation des DTOs

Au départ, nous exposions directement les entités JPA dans nos réponses API. Cela a généré :

- Des problèmes de sérialisation JSON avec boucles infinies,
- Des erreurs `LazyInitializationException`.

La création de DTOs nous a permis :

- De maîtriser la structure des réponses,
 - De sécuriser les données exposées,
 - De transformer les entités en objets adaptés à la vue (ex. formatage du temps d'attente).
-

Architecture frontend

L'application frontend repose sur :

- Des **composants Svelte** : Vue patient, tableau de bord infirmier, carte, formulaire...
- Des **Stores Svelte** : pour partager l'état global (utilisateur connecté, patient sélectionné...).
- Des **interfaces TypeScript** : garantissant la cohérence avec les DTOs backend.
- Un **module api.ts** : centralisant les appels API (fetch + gestion erreurs).

Modélisation Conceptuelle

Entités

1. Hôpital

- Attributs: IDHopital , NomHopital , AdrHopital , Longitude , Latitude
- Description: Chaque hôpital est identifié de manière unique et possède une adresse. Cette entité est essentielle pour localiser géographiquement les établissements sur la carte interactive.

2. Département

- Attributs: numDep , nomDep
- Description: Les hôpitaux sont divisés en départements (par exemple : urgences, pédiatrie, etc.). Chaque département appartient à un seul hôpital.

3. Salle d'attente

- Attributs: IDSalle , nomSalle
- Description: Chaque département peut avoir une salle d'attente, identifiée par son ID unique et son nom.

4. Personne

- Attributs: IDPersonne , nom , prenom , numTel
- Description: Représente toute personne interagissant avec le système, que ce soit un patient ou un infirmier.

5. Patient (spécialisation de Personne)

- Attributs: numéroAssurance , adresse
- Description: Représente une personne qui utilise le service d'attente. Chaque patient a un numéro d'assurance maladie unique.

6. Infirmier (spécialisation de Personne)

- Attributs: matricule , password
- Description: L'infirmier est distingué par son rôle dans l'évaluation du patient. Il a un matricule et un mot de passe pour accéder au système.

7. Évaluation Médicale

- Attributs: IDEvaluation , sévérité , commentaire , timestampEvaluation
- Description: Permet d'enregistrer une évaluation médicale d'un patient , en associant un degré de sévérité au patient et en enregistrant le moment de l'évaluation.

8. Inscription alerte

- Attributs: notificationEnvoyee , contactNotification , IDInscription , timestampInscription
- Description: Permet à un utilisateur de s'inscrire aux alertes qui seront envoyées au contact indiqué. Chaque utilisateur reçoit un identifiant unique et le moment de l'inscription est enregistré.

9. Systeme alerte config

- Attributs: IDConfigAlerte , Méthode , SeuilAlerte
- Description: Permet à un utilisateur inscrit de configurer les alertes, en réglant le nombre de patients qui déclenche l'alerte et la méthode d'envoi.

10. Sejour Salle Attente

- Attributs: IDSejour , Arrivee , Depart
- Description: Représente la période dans laquelle un patient se trouve en salle d'attente, en identifiant chaque séjour unique avec un ID et marquant le moment du début et de la fin du séjour.

Relations

1. Possède

- Entités: Hôpital $\rightarrow$ Département
- Cardinalité: 1:n
- Description: Un hôpital peut posséder plusieurs départements. Un département appartient à un seul hôpital.

2. Gère

- Entités: Département $\rightarrow$ Salle d'attente
- Cardinalité: 1:1 $\leftrightarrow$ 0:n
- Description: Chaque salle d'attente est gérée par un seul département. Un département peut gérer plusieurs salles d'attente.

3. A lieu

- Entités: SejourSalleAttente $\leftrightarrow$ Salle d'attente
- Cardinalité: 0:n $\leftrightarrow$ 1:1
- Description: Un séjour se déroule dans une seule salle d'attente spécifique. Une salle d'attente peut accueillir plusieurs séjours différents.

4. Attends

- Entités: Patient $\leftrightarrow$ SejourSalleAttente
- Cardinalité: 1:1 $\leftrightarrow$ 0:n
- Description: Un patient peut avoir plusieurs séjours en salle d'attente (à différents moments). Chaque séjour correspond à un seul patient.

5. A eval

- Entités: SejourSalleAttente $\leftrightarrow$ ÉvaluationMédicale
- Cardinalité: 0:1 $\leftrightarrow$ 1:1
- Description: Un séjour peut être associé à une évaluation médicale (facultatif). Une évaluation médicale est liée à un séjour spécifique.

6. Concerne

- Entités: Évaluation Médicale $\rightarrow$ Patient
- Cardinalité: 1:1 $\leftrightarrow$ 1:n
- Description: Une évaluation médicale concerne un seul patient, mais un patient peut avoir plusieurs évaluations au cours du temps.

7. Effectue

- Entités: Infirmier $\rightarrow$ Évaluation Médicale
- Cardinalité: 1:n $\leftrightarrow$ 1:1
- Description: Une évaluation médicale est effectuée par un seul infirmier, mais un infirmier peut en effectuer plusieurs.

8. S'inscrit

- Entités: SejourSalleAttente $\leftrightarrow$ InscriptionAlerte
- Cardinalité: 0:1 $\leftrightarrow$ 1:1
- Description: Une inscription d'alerte est liée à un séjour spécifique (facultatif), permettant au patient de recevoir des notifications concernant ce séjour particulier.

9. Configure

- Entités: Salle d'attente $\leftrightarrow$ SystemeAlerteConfig
- Cardinalité: 0:1 $\leftrightarrow$ 1:1

- Description: Une salle d'attente peut être configurée avec un système d'alerte spécifique (facultatif). Un système d'alerte est configuré pour une seule salle d'attente.

Modèle Logique Relationnel

Schéma Relationnel

1. **Hôpital** : Hôpital(*IDHopital*, nomHopital, adrHopital, latitude, longitude)
2. **Département** : Departement(*NumDep*, nomDep, #IDHopital)
3. **Salle d'attente** : SalleAttente(*IDSalle*, nomSalle, #NumDep)
4. **Personne (NC)** : Personne(*IDPersonne*, nom, prenom, numTel)
5. **Infirmier (NC)** : Infirmier(*IDPersonneInfirmier*, #IDPersonne, matricule, password)
6. **Patient (NC)** : Patient(*numAssurance*, adresse, #IDPersonne)
7. **Évaluation Médicale** : EvaluationMedicale(*IDEvaluation*, severite, commentaire, timestampEvaluation, #numAssurance, #IDPersonneInfirmier)
8. **SystemeAlerteConfig (NC)** : SystemeAlerteConfig(*IDConfigAlerte*, #IDSalle, Methode, SeuilAlerte)
9. **Inscription (NC)** : Inscription(*IDInscription*, #IDSejour, timestampInscription, notificationEnvoyee, contactNotification, methodeChoisie)
10. **SejourSalleDattente** : sejourSalleDattente(*IDSejour*, #IDSalle, #numAssurance, arrivée, départ, #IDEvaluation)

Normalisation

Dépendances Fonctionnelles

1. Hôpital

- DF: IDHopital -> nomHopital , adrHopital , latitude , longitude
- Justification: IDHopital est unique donc il va déterminer nom, adresse, latitude et longitude.

2. Département

- DF: NumDep -> nomDep , #IDHopital
- Justification: Chaque département a un id unique, cela va déterminer le nom du département et dans quel hôpital il se situe.

3. Salle d'attente

- DF: IDSalle -> #NumDep , nomSalle
- Justification: Chaque salle d'attente a un id unique qui va permettre de déterminer dans quel département (#NumDep) elle se situe et son nom.

4. Personne (NC)

- DF: IDPersonne -> nom , prenom , numTel
- Justification: Chaque personne est identifiée par un IDPersonne unique, qui détermine les données de la personne (nom, prénom, # de téléphone) qui ne sont pas liées entre elles.

5. Infirmier (NC)

- DF: IDPersonneInfirmier -> #IDPersonne , matricule , password
- Justification: La clé IDPersonneInfirmier fait référence à l'identifiant de la personne et son matricule d'employé et son mot de passe au système. Le mot de passe pourrait être associé au matricule directement, mais on l'associe plutôt à l'identifiant unique.

6. Patient (NC)

- DF: numAssurance -> adresse , #IDPersonne
- Justification: Le numéro d'assurance unique identifie la personne par son ID et l'adresse.

7. Évaluation Médicale

- DF: IDEvaluation -> severite , commentaire , timestampEvaluation , #numAssurance , #IDPersonneInfirmier
- Justification: IDEvaluation est un id unique qui va déterminer severite, commentaire, timestampEvaluation, numAssurance, IDPersonneInfirmier les autres relations entre elles n'ont pas de DF car elles sont variables, par exemple un infirmier peut donner plusieurs sévérité ou traiter plusieurs numéros d'assurances.

8. SystemeAlerteConfig (NC)

- DF: IDConfigAlerte -> #IDSalle , Methode , SeuilAlerte
- Justification: La clé simple IDConfigAlerte détermine directement le numéro de la salle, le seuil et la méthode. Les autres attributs sont des éléments distincts de la configuration de l'alerte, ils n'ont donc aucune dépendance entre eux.

9. Inscription (NC)

- DF: IDInscription -> #IDSejour , timestampInscription , notificationEnvoyee , contactNotification , methodeChoisie
- Justification: IDInscription gère chaque attribut de façon unique. Il capture toutes les variations possibles (e.g. un changement de contact), donc toute autre DF serait triviale.

10. SejourSalleDattente

- DF: IDSejour -> #IDSalle , #numAssurance , arrivée , départ , #IDEvaluation

- Justification: La clé IDsejour sera relier a chacun des autres attributs de facon unique. Celle ci est la clé primaire donc elle sera toujours unique.

Processus de Normalisation

1. **Hôpital:** 1NF (atomique), 2NF (clé simple), 3NF (pas de dépendance transitive), BCNF (clé simple). Déjà normalisée.
2. **Département:** 1NF (atomique), 2NF (clé simple), 3NF (pas de dépendance transitive), BCNF (clé simple). Déjà normalisée.
3. **SalleAttente:** 1NF (atomique), 2NF (clé simple), 3NF (pas de dépendance transitive), BCNF (clé simple). Déjà normalisée.
4. **Personne:** 1NF (atomique), 2NF (clé simple), 3NF (pas de dépendance transitive), BCNF (clé simple). Déjà normalisée.
5. **Infirmier:** 1NF (atomique), 2NF (clé simple), 3NF (pas de dépendance transitive), BCNF (clé simple). Déjà normalisée.
6. **Patient:** 1NF (atomique), 2NF (clé simple), 3NF (pas de dépendance transitive), BCNF (clé simple). Déjà normalisée.
7. **EvaluationMedicale:** 1NF (atomique), 2NF (clé simple), 3NF (pas de dépendance transitive), BCNF (clé simple). Déjà normalisée.
8. **SystemeAlerteConfig:** 1NF (atomique), 2NF (clé simple), 3NF (pas de dépendance transitive), BCNF (clé simple). Déjà normalisée.
9. **Inscription:** 1NF (atomique), 2NF (clé simple), 3NF (pas de dépendance transitive), BCNF (clé simple). Déjà normalisée.
10. **sejourSalleDattente:** 1NF (atomique), 2NF (clé simple), 3NF (pas de dépendance transitive), BCNF (clé simple). Déjà normalisée.

Schéma Final Normalisé

1. **Hôpital:** Hôpital(*IDHopital*, nomHopital, adrHopital, latitude, longitude)
2. **Département:** Departement(*NumDep*, nomDep, #IDHopital)
3. **Salle d'attente:** SalleAttente(*IDSalle*, #NumDep)
4. **Personne:** Personne(*IDPersonne*, nom, prenom, numTel)
5. **Infirmier:** Infirmier(*IDPersonneInfirmier*, #IDPersonne, matricule, password)
6. **Patient:** Patient(*numAssurance*, adresse, #IDPersonne)
7. **Évaluation Médicale:** EvaluationMedicale(*IDEvaluation*, severite, #numAssurance, #IDPersonneInfirmier)
8. **SystemeAlerteConfig:** SystemeAlerteConfig(*IDConfigAlerte*, #IDSalle, Methode, SeuilAlerte)
9. **Inscription:** Inscription(*IDInscription*, #IDSejour, timestampInscription, notificationEnvoyee, contactNotification, methodeChoisie)
10. **SejourSalleDattente:** sejourSalleDattente(*IDSejour*, #IDSalle, #numAssurance, arrivee, depart, #IDEvaluation)

Exemples de Requêtes pour notre interface (Q&A)

Question 1: Quel est le temps d'attente moyen par catégorie de sévérité ?

Algèbre Relationnelle

```
r1 ← π idevaluation, arrivee, depart (σ depart IS NOT NULL (SejourSalleAttente))
r2 ← π idevaluation, severite (EvaluationMedicale)
r3 ← r1 ⋈ idevaluation = idevaluation (r2)
ρ(severite, tempsAttenteMoyen) (
    severite A avg(depart – arrivee)(r3))
```

SQL

```
SELECT
    e.severite,
    AVG(EXTRACT(EPOCH FROM (s.depart - s.arrivee)) / 60) AS temps_attente_moyen_minutes
FROM sejoursalleattente s
JOIN evaluationmedicale e
    ON s.idevaluation = e.idevaluation
WHERE s.depart IS NOT NULL
GROUP BY e.severite;
```

Optimisation

- On filtre en amont (depart IS NOT NULL) pour ne traiter que les séjours terminés, réduisant la taille du jeu de données.
 - On projette uniquement idevaluation, arrivee et depart avant la jointure, limitant le volume de données manipulées.
 - La jointure ne ramène que la colonne severite depuis EvaluationMedicale, évitant le chargement de colonnes non pertinentes.
 - Le calcul avg(depart – arrivee) est effectué directement par le SGBD en une seule passe, tirant parti d'éventuels index temporels.
-

Question 2: Quel est le temps d'attente moyen pour chaque salle d'attente dans chaque département ?

Algèbre Relationnelle

```
r1 ← π idsalle, arrivee, depart (σ depart IS NOT NULL (SejourSalleAttente))
r2 ← π idsalle, numdep (SalleAttente ⋈ Departement)
r3 ← r1 ⋈ idsalle = idsalle (r2)
p(numdep, idsalle, tempsAttenteMoyen) (
    (numdep, idsalle) A avg(depart - arrivee)(r3))
```

SQL

```
SELECT
    sa.numdep                                AS num_departement,
    s.idsalle                                AS id_salle,
    AVG(EXTRACT(EPOCH FROM (s.depart - s.arrivee)) / 60)
        AS temps_attente_moyen_minutes
FROM sejoursalleattente s
JOIN salleattente sa
    ON s.idsalle = sa.idsalle
WHERE s.depart IS NOT NULL
GROUP BY sa.numdep, s.idsalle;
```

Optimisation

- On applique d'abord la restriction depart IS NOT NULL pour exclure les séjours actifs.
 - On ne joint que SalleAttente pour récupérer numdep, minimisant le nombre de jointures.
 - La projection sur idsalle et numdep avant l'agrégation réduit le volume de données à grouper.
 - L'agrégation groupée par numdep et idsalle est optimisée par les index sur idsalle et depart.
-

Question 3 : Liste des hôpitaux avec le nombre de patients en attente et le temps d'attente moyen actuel (en minutes)

Algèbre Relationnelle

```
H ← π idHopital, nomHopital (Hopital)
D ← π numDep, idHopital (Departement)
SA ← π idSalle, numDep (SalleAttente)
S ← σ depart IS NULL (SejourSalleAttente)
T ← S ⋈ idSalle = idSalle (SA) ⋈ numDep = numDep (D) ⋈ idHopital = idHopital (H)
T1 ← π idHopital, nomHopital, arrivee (T)
T2 ← π idHopital, nomHopital, EXTRACT(EPOCH FROM (now() - arrivee)) AS Tsec (T1)
ρ(idHopital, nomHopital, qtePatientsEnAttente, tempsAttenteMoyenne) (
    (idHopital, nomHopital) A (count(arrivee), ceil(avg(Tsec/60))) (T2))
```

SQL

```
SELECT h.idhopital, h.nomhopital, COUNT(s.idsejour) AS qte_patients_en_attente,
CEIL(AVG(EXTRACT(EPOCH FROM (NOW() - s.arrivee)) / 60)) AS temps_attente_moyenne_minutes
FROM hopital h
LEFT JOIN departement d
    ON d.idhopital = h.idhopital
LEFT JOIN salleattente sa
    ON sa.numdep = d.numdep
LEFT JOIN sejoursalleattente s
    ON s.idsalle = sa.idsalle
    AND s.depart IS NULL
GROUP BY h.idhopital, h.nomhopital;
```

Optimisation

- LEFT JOIN pour inclure les hôpitaux sans patients en attente.
 - Filtrage s.depart IS NULL directement dans le JOIN pour réduire le volume traité.
 - Agrégations (COUNT, AVG) groupées sur l'hôpital, tirant parti des index sur departement.idhopital et sejoursalleattente.idsalle.
 - Calcul de la durée (NOW() - arrivee) déporté au SGBD pour bénéficier d'optimisations natives.
-

Question 4: Combien de patients en attente par niveau de sévérité pour un département donné ?

Algèbre Relationnelle

```
S ← σ depart IS NULL (SejourSalleAttente)
SA ← π idSalle, numDep      (SalleAttente)
E ← π idevaluation, severite (EvaluationMedicale)
T ← S ⋈ idSalle = idSalle (SA)
R ← T ⋈ idevaluation = idevaluation (E)
ρ(numDep, severite, qtePatients) (
    (numDep, severite) A count(idsejour) (π numDep, severite (R)))
```

SQL

```
SELECT sa.numdep AS num_departement,
       COALESCE(em.severite, 0) AS severite,
       COUNT(*) AS qte_patients_en_attente
FROM sejoursalleattente s
JOIN salleattente sa
  ON s.idsalle = sa.idsalle
LEFT JOIN evaluationmedicale em
  ON s.idevaluation = em.idevaluation
WHERE s.depart IS NULL
      AND sa.numdep = :numDep
GROUP BY sa.numdep, em.severite;
```

Optimisation

- Projection de idsalle, numdep et idevaluation avant les jointures pour limiter la taille des relations.
- Filtre s.depart IS NULL pour ne conserver que les séjours actifs.
- LEFT JOIN sur evaluationmedicale pour inclure les patients non évalués (sévérité = 0).
- Agrégation groupée sur numdep et severite, exploitant les index sur salleattente.numdep et sejoursalleattente.depart.

Défis rencontrés

- Boucles infinies JSON entre entités liées (résolues avec @JsonIgnore, DTOs).
- Chargement lazy vs eager : corrigé avec JOIN FETCH.
- Séparation claire des flux métier (inscription vs évaluation).
- Gestion des états croisés entre composants Svelte (résolue par une coordination fine des stores).
- Collaboration d'équipe : uniformisation du code, review des branches, coordination GitHub.

Coordination d'équipe

Notre équipe de 4 personnes a travaillé sur une période de 3 semaines. On a dû s'organiser pour:

- Le découpage des tâches,
- La mise en place de la structure du dépôt,
- L'intégration continue avec Git,
- Le choix des outils (Spring Boot, Svelte, PostgreSQL...),
- Et la synchronisation des modules frontend/backend.

Mini-guide utilisateur

1. Selection du rôle de l'utilisateur

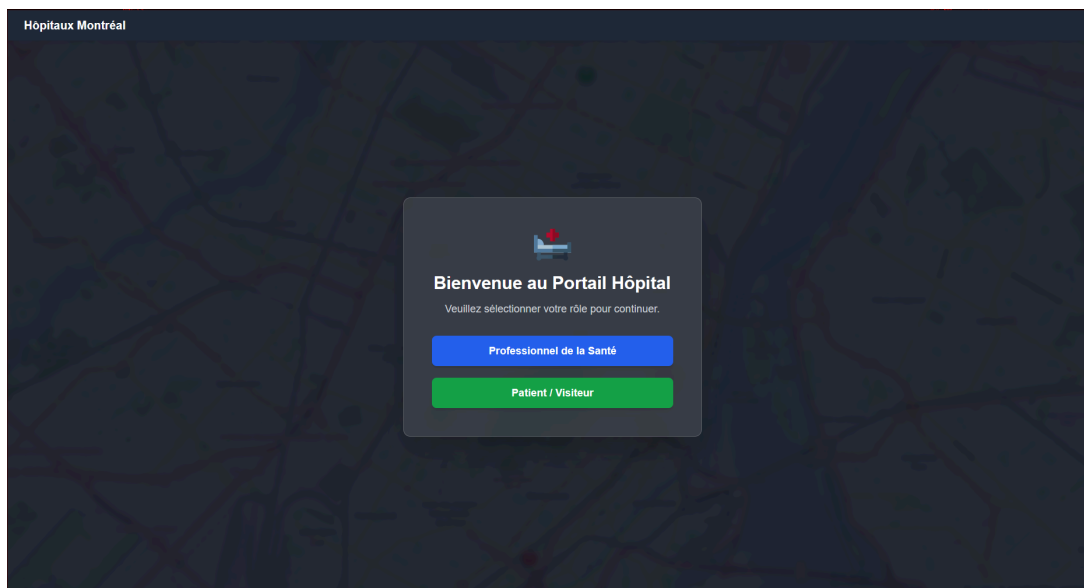


Figure 1: Choisir le rôle correspondant en cliquant sur le bouton

2. Utiliser un compte test pour se connecter

2.1 Côté infirmier

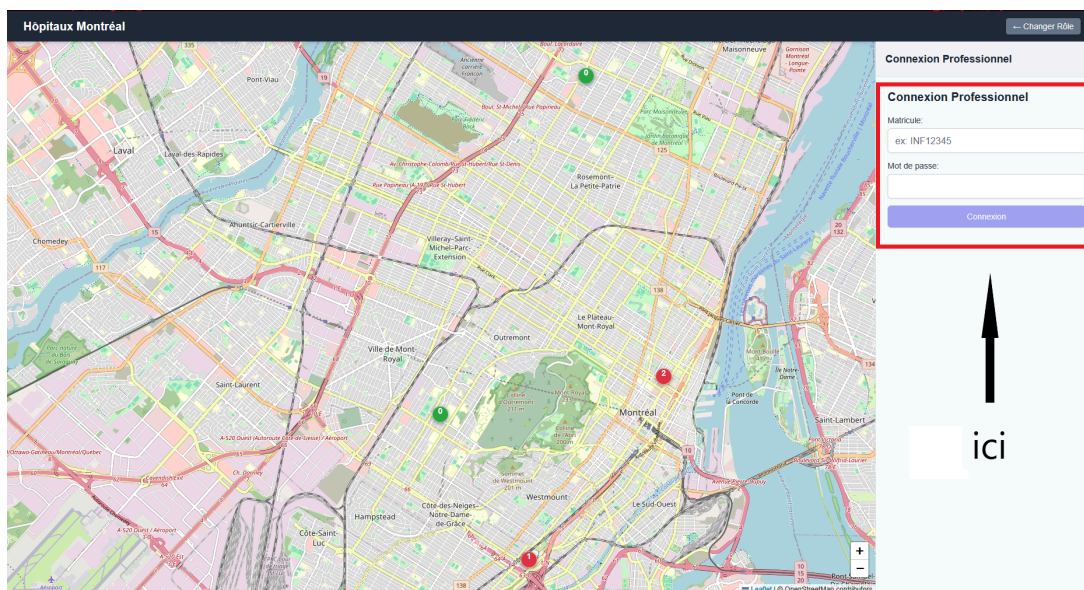


Figure 2: Choisir un compte test et se connecter

- **Infirmier CHUM**
 - Matricule : INF987CHUM
 - Mot de passe : julie123
- **Infirmier CUSM**
 - Matricule : INF123CUSM
 - Mot de passe : pierre456

2.1 Côté patient ou visiteur

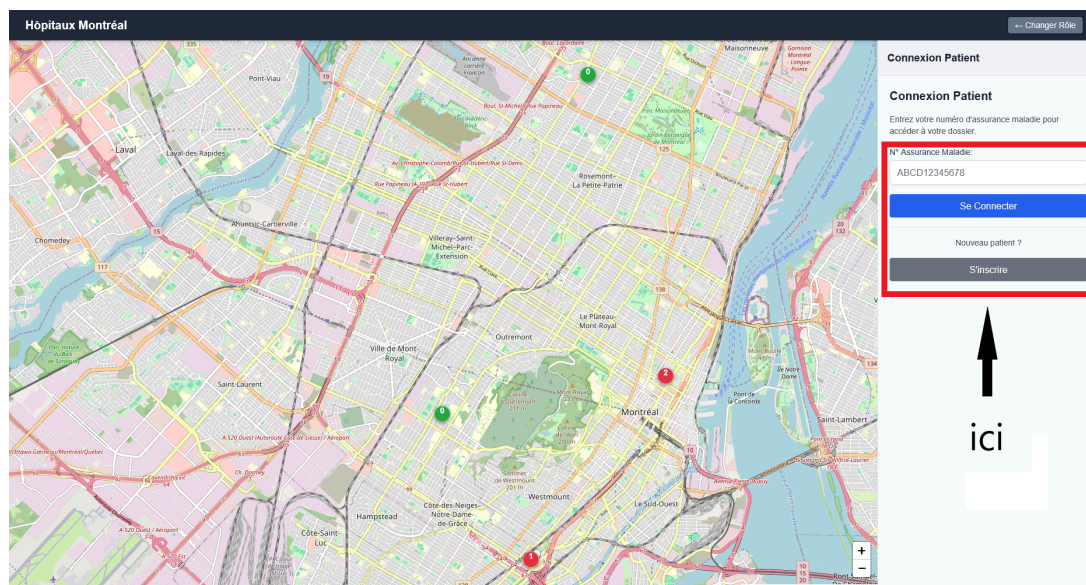


Figure 3: Choisir un compte test et se connecter à l'aide de la RAMQ

Vous pouvez aussi vous inscrire en tant que visiteur, sans compte, préalable. Or, vous n'aurez pas les mêmes fonctionnalités qu'un patient. Vous devez d'abord vous faire évaluer (condition réelle basée sur la vraie vie.)

- **Patient:** Jean Tremblay
 - RAMQ : TRAMJ12345678
- **Patient:** Marie Gagnon
 - RAMQ : GAGNM010170123
- **Patient:** John Smith
 - RAMQ : SMITJ010280456

3. Dashboard principal du personnel infirmier

3.1 Page principale

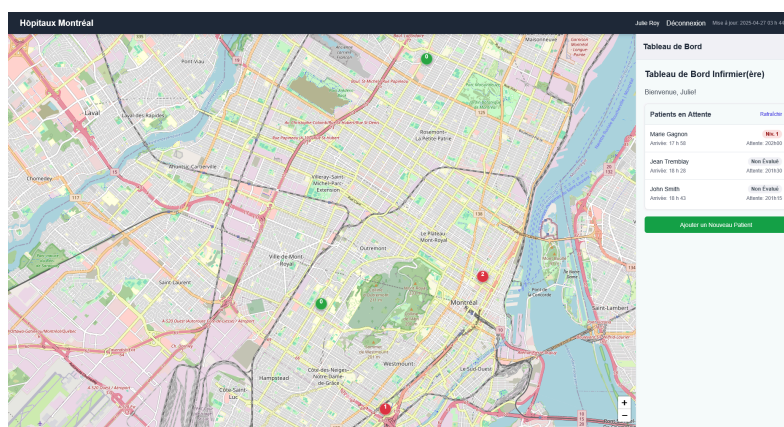


Figure 4: Tableau de bord avec patients triés par sévérité

Vous pouvez maintenant accéder aux fonctionnalités du personnel infirmier.

- Tableau de bord avec patients triés par sévérité
- Consultation de chaque patient

3.2 Évaluation médicale - côté dashboard infirmier

Évaluation Patient

Évaluer:
GAGNM010170123 [< Retour Dashboard](#)

Arrivé le: 18 avr. 2025 17 h 58

Niveau de Sévérité (1-5):
1 (Non urgent) ▼

Commentaires / Symptômes:
Entrez les détails cliniques pertinents...

Enregistrer Évaluation

Figure 5: Évaluation du patient (niveau de sévérité, commentaire)

- Détail spécifique au patient choisit
- Ajout ou modification d'affectation de sévérité pour un patient.
- Ajout ou modification de commentaire sur l'état du patient

3.3 Enregistrement d'un nouveau patient

Inscription Patient (Infirmier)

Inscription Patient [< Annuler / Retour](#)

Nom:

Prénom:

N° Assurance Maladie:
ABCD12345678 ou autre

N° Téléphone:
514...

Adresse (optionnel):

Niveau de Sévérité (Triage Initial) *
-- Choisir un niveau -- ▼

Commentaire Triage (optionnel):

Salle d'Attente (Simul.):
-- Choisir une salle -- ▼

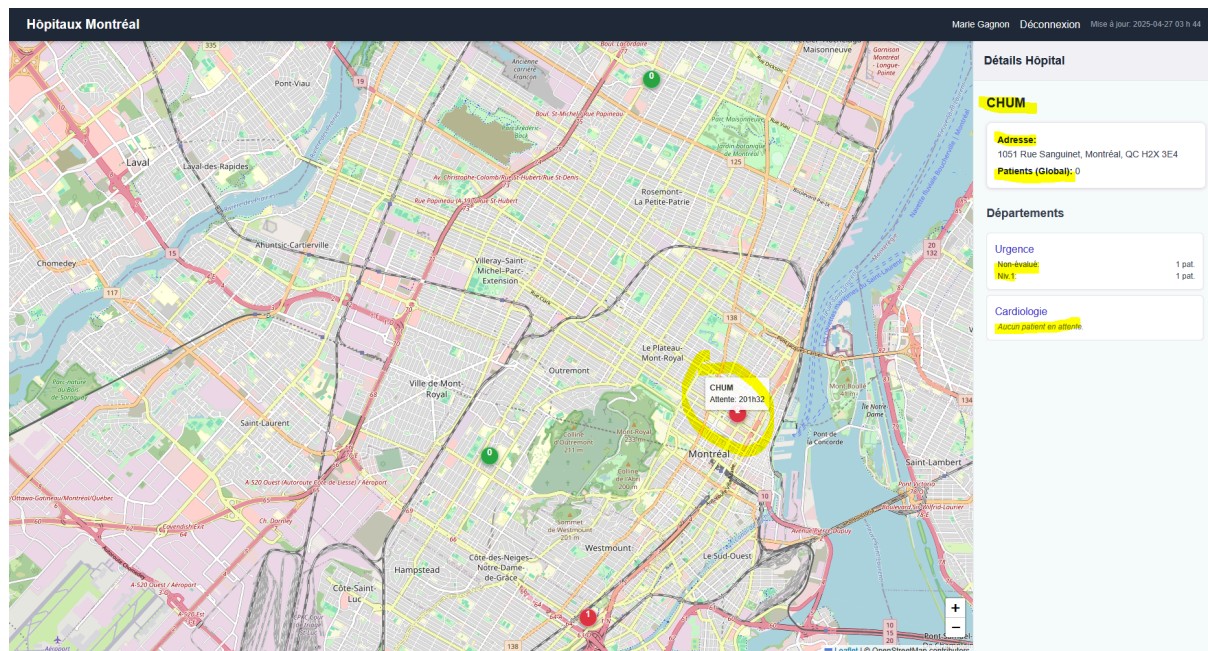
Enregistrer le Patient

Figure 6: Enregistrement d'un nouveau patient (niveau de sévérité, commentaire, adresse etc)

- Simulation réelle d'enregistrement d'un patient comme lors du triage dans la vraie vie.
- Affectation de la salle correspondante au patient.
- Possibilité de modifier le rapport depuis le dashboard après l'enregistrement

4. Dashboard principal du patient

4.1 Page principale



Marqueurs interactifs

- Hôpitaux affichés sous forme de points cliquables sur la carte
- Couleurs ou icônes indiquant l'état global (ex : vert pour 0 patient, rouge si attente)
- **Pop-up d'information**
 - Au clic sur un hôpital, affiche un encart sur la carte
 - Affiche le nom de l'hôpital et le temps d'attente moyen estimé (ex : "Attente : 201 h 32")
- **Panneau "Détails Hôpital"**
 - Nom de l'hôpital en titre
 - Adresse complète
 - Nombre total de patients en attente (global)
- **Liste des départements**
 - Pour chaque département du même hôpital :
 - État : "Aucun patient en attente" ou décompte par sévérité (ex : "Non-évalué : 1 pat.", "Niv. 1 : 1 pat.")
 - Affiche "0 pat." si aucune attente
- **Mise à jour en temps réel**
 - Les marqueurs et le panneau se rafraîchissent automatiquement lorsque les données de séjour changent

4.2 Informations générales

The screenshot displays a patient dashboard with the following sections:

- Mon Séjour Actif** (Header)
- Mon Statut Actuel** (Section Header)
 - Patient:** Marie Gagnon
 - N° Assurance:** GAGNM010170123
 - Téléphone:** 4389998877
 - Adresse:** 456 Avenue Elm, Laval
- Séjour Actif** (Section Header)
 - Hôpital:** CHUM
 - Salle:** Salle Urgence Principale
 - Arrivée:** 17 h 58
 - Temps d'attente:** 202h12
 - Évaluation:** Niv. 1
 - Sévérité:** 1
- Notifications SMS** (Section Header)
 - Configurez une alerte SMS pour être notifié(e) lorsqu'il restera un certain nombre de patients avant vous. Le numéro de téléphone utilisé sera celui enregistré dans votre profil (modifiable dans la section ci-dessus).
 - M'alerter quand il reste: patient(s) avant moi
 - Inscrire / Mettre à jour** (Blue button)
 - Me désinscrire** (Red button)

At the bottom, a green bar shows the status: **Connexion réussie.**

Figure 7: Dashboard principale de la vue patient

- **Mon Statut Actuel**
 - Affiche le nom complet du patient et son numéro d'assurance
 - Affiche le téléphone et l'adresse du patient
- **Séjour Actif**
 - Nom de l'hôpital et de la salle d'attente
 - Heure d'arrivée formatée (ex : "17 h 58")
 - Temps d'attente écoulé (ex : "202 h 12" – heures et minutes)
 - Statut d'évaluation avec niveau de sévérité (ex : "Niv. 1")
 - Valeur numérique de la sévérité
- **Notifications SMS**
 - Zone de configuration expliquant le principe de l'alerte
 - Sélecteur du nombre de patients restant devant soi (ex : 5)
 - Bouton "Inscrire / Mettre à jour" pour (re)configurer l'alerte
 - Bouton "Me désinscrire" pour annuler l'alerte
- **Feedback visuel**
 - Message de confirmation "Connexion réussie." en bas de l'écran
 - Encarts colorés pour distinguer les sections (gris clair, bleu, vert pâle)

Conclusion

Le projet **Système de Gestion de Salle d'Attente Hospitalière** constitue une application web complète, utilisant une architecture en couches moderne et des outils technologiques robustes. Il nous a permis :

- D'appliquer concrètement les notions de modélisation relationnelle,
- De travailler en contexte réel sur un projet full-stack,
- De résoudre des problèmes concrets liés à la sérialisation, la persistance et la sécurité,
- Et de consolider notre autonomie technique et notre capacité à collaborer efficacement.

Projet réalisé entre mars et avril 2025.